UNIT I: Review of Operating Systems: Basic Principles, system calls, Files-Processes, design and implementation of processes, Communication between processes, operating system structures.

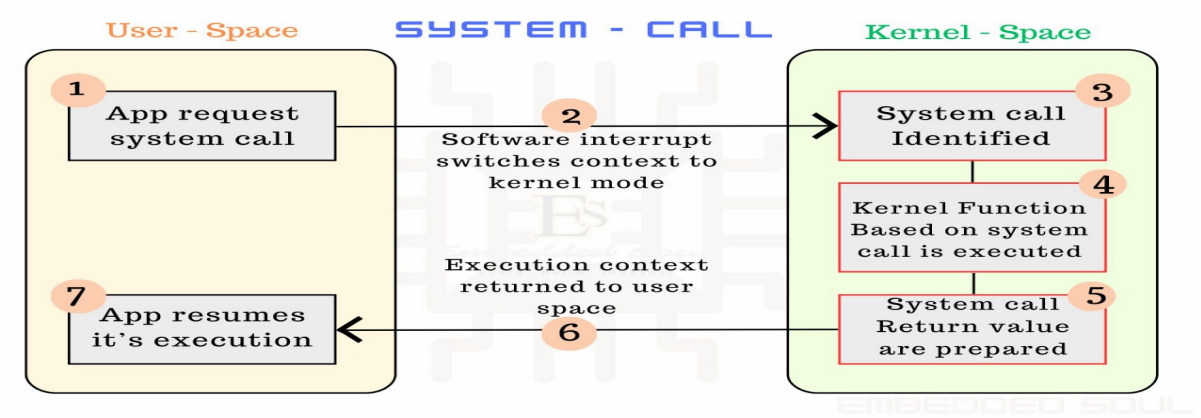
1. Basic Principles of Operating Systems:

An Operating System (OS) is system software that manages computer hardware and software resources and provides services to application programs. It acts as an interface between the user and the computer hardware.

Objectives of OS: include convenience, efficiency, security, reliability, and scalability.
The OS ensures effective utilization of resources such as CPU, memory, and I/O devices.
Major functions of an OS include process management, memory management, file system management, I/O management, secondary storage management, and protection mechanisms.

2. System Calls:

System calls provide an interface to the services made available by an operating system. These calls are generally available as routines written in C and C++, although certain low-level tasks (for example, tasks where hardware must be accessed directly) may have to be written using assembly-language instructions. An example to illustrate how system calls are used: writing a simple program to read data from one file and copy them to another file. Frequently, systems execute thousands of system calls per second. Most programmers never see this level of detail, however. Typically, application developers design programs according to an application programming interface (API). The API specifies a set of functions that are available to an application programmer, including the parameters that are passed to each function and the return values the programmer can expect. Three of the most common APIs available to application programmers are the Windows API for Windows systems, the POSIX API for POSIX-based systems (which include virtually all versions of UNIX, Linux, and Mac OSX), and the Java API for programs that run on the Java virtual machine. A programmer accesses an API via a library of code provided by the operating system. In the case of UNIX and Linux for programs written in the C language, the library is called libc. Behind the scenes, the functions that make up an API typically invoke the actual system calls on behalf of the application programmer. For example, the Windows function CreateProcess() (which unsurprisingly is used to create a new process) actually invokes the NTCreateProcess() system call in the Windows kernel.



For most programming languages, the run-time support system (a set of functions built into libraries included with a compiler) provides a system call interface that serves as the link to system calls made available by the operating system. The system-call interface intercepts function calls in the API and invokes the necessary system calls within the operating system.

Types of System Calls:

• Process control

• File management

• Device management

• Information maintenance

• Communications

• Protection

Process Control :

A running program needs to be able to halt its execution either normally (end()) or abnormally (abort()). If a system call is made to terminate the currently running program abnormally, or if the program runs into a problem and causes an error trap, a dump of memory is sometimes taken and an error message generated. The dump is written to disk and may be examined by a debugger—a system program designed to aid the programmer in

finding and correcting errors, or bugs—to determine the cause of the problem. Under either normal or abnormal circumstances, the operating system must transfer control to the invoking command interpreter. The command interpreter then reads the next command.

	Windows	Unix
Process Control	CreateProcess()	fork()
	ExitProcess()	exit()
	WaitForSingleObject()	wait()
File Manipulation	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()
Device Manipulation	SetConsoleMode()	ioctl()
	ReadConsole()	read()
	WriteConsole()	write()
Information Maintenance	GetCurrentProcessID()	getpid()
	SetTimer()	alarm()
	Sleep()	sleep()
Communication	CreatePipe()	pipe()
	CreateFileMapping()	shmget()
	MapViewOfFile()	mmap()
Protection	SetFileSecurity()	chmod()
	InitializeSecurityDescriptor()	umask()
	SetSecurityDescriptorGroup()	chown()

To start a new process, the shell executes a fork() system call. Then, the selected program is loaded into memory via an exec() system call, and the program is executed. Depending on the way the command was issued, the shell then either waits for the process to finish or runs the process "in the background."

File Management:

We first need to be able to create() and delete() files. Either system call requires the name of the file and perhaps some of the file's attributes. Once the file is created, we need to open() it and to use it. We may also read(), write(), or reposition() (rewind or skip to the end of the file, for example). Finally, we need to close() the file, indicating that we are no longer using it.

Device Management:

Process may need several resources to execute—main memory, disk drives, access to files, and so on. If the resources are available, they can be granted, and control can be returned to the user process. Otherwise, the process will have to wait until sufficient resources are available. The various resources controlled by the operating system can be thought of as devices. Some of these devices are physical devices (for example, disk drives), while others can be thought of as abstract or virtual devices (for example, files).A system with multiple users may require us to first request() a device, to ensure exclusive use of it. After we are finished with the device, we release() it. Once the device has been requested (and allocated to us), we can read(), write(), and (possibly) reposition() the device, just as we can with files.

Information Maintenance:

Many system calls exist simply for the purpose of transferring information between the user program and the operating system. For example, most systems have a system call to return the current time() and date(). Other system calls may return information about the system, such as the number of current users, the version number of the operating system, the amount of free memory or disk space, and so on. Another set of system calls is helpful in debugging a program. Many systems provide system calls to dump() memory. This provision is useful for debugging.

Communication:

There are two common models of inter process communication: the message passing model and the shared-memory model. In the message-passing model, the communicating processes exchange messages with one another to transfer information. Messages can be exchanged between the processes either directly or indirectly through a common mailbox. Before communication can take place, a connection must be opened.

Protection:

Protection provides a mechanism for controlling access to the resources provided by a computer system. Typically, system calls providing protection include set permission() and get permission(), which manipulate the permission settings of resources such as files and disks. The allow user() and deny user() system calls specify whether particular users can—or cannot—be allowed access to certain resources.

3. Files and Processes:

3.1 Files:

3.1 Definition of a File

A file is a collection of related information stored on secondary storage (such as hard disk or SSD) and managed by the operating system. Files provide a way to store data permanently.

3.2 File Characteristics

Files are stored on secondary storage

Files survive system shutdown

Files are organized in a file system

Access to files is controlled by the OS

3.3 File Attributes

Each file has metadata called file attributes, which include:

Name – Human-readable file name

Identifier – Unique number (file ID)

Type – Text, binary, executable, etc.

Location – Pointer to file location on disk

Size – Current file size

Protection – Read, write, execute permissions

Time & Date – Creation, modification, access time

3.4 File Types

Text files – Human-readable characters

Binary files – Executable or machine-readable

Source files – Program source code

Object files – Compiled code

Executable files – Ready to run programs

3.5 File Operations

The operating system supports the following file operations:

Create – Creates a new file

Open – Opens file for access

Read – Reads data from file

Write – Writes data to file

Append – Adds data at end of file

Seek – Changes file pointer position

Close – Closes file and releases resources

Delete – Removes file permanently

3.6 File Access Methods

1. Sequential Access

Data accessed in order

Example: Text files

Operations: read next, write next

2. Direct (Random) Access

Data accessed using file position

Faster than sequential access

Example: Databases

3. Indexed Access

Index contains pointers to data blocks

Combines sequential and direct access

Used in large file systems

3.7 File Structure

Unstructured – Sequence of bytes

Structured – Records or fields

OS usually treats files as unstructured for flexibility

3.8 File Protection

Protection ensures that only authorized users can access files.

Access Rights

Read (R)

Write (W)

Execute (X)

Protection Mechanisms

Access control lists (ACLs)

User/group permissions

Password protection

3.2 Processes:

A process, which is a program in execution. A process is the unit of work in a modern time-sharing system. A system therefore consists of a collection of processes: operating system processes executing system code and user processes executing user code. Potentially, all these processes can execute concurrently, with the CPU (or CPUs) multiplexed among

them. By switching the CPU between processes, the operating system can make the computer more productive.

A process is also known as job or task. A process is more than the program code, which is sometimes known as the text section. It also includes the current activity, as represented by the value of the program counter and the contents of the processor's registers. A process generally also includes the process stack, which contains temporary data (such as function parameters, return addresses, and local variables), and a data section, which contains global variables. A process may also include a heap, which is memory that is dynamically allocated during process run time. The structure of a process in memory is shown in Figure 3.1.

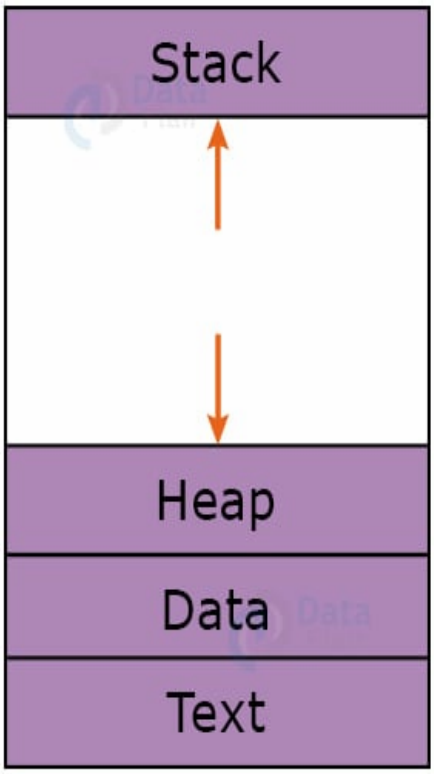


Fig 3.1

We emphasize that a program by itself is not a process. A program is a passive entity, such as a file containing a list of instructions stored on disk (often called an executable file). In contrast, a process is an active entity, with a program counter specifying the next instruction to execute and a set of associated resources. A program becomes a process when an executable file is loaded into memory.

Process State:

As a process executes, it changes state. The state of a process is defined in part by the current activity of that process.

A process may be in one of the following states:

• New. The process is being created.

• Running. Instructions are being executed.

• Waiting. The process is waiting for some event to occur (such as an I/O completion or reception of a signal).

• Ready. The process is waiting to be assigned to a processor.

• Terminated. The process has finished execution. It is important to realize that only one process can be running on any processor at any instant.

Many processes may be ready and waiting, however. The state diagram corresponding to these states is presented in Figure 3.2.

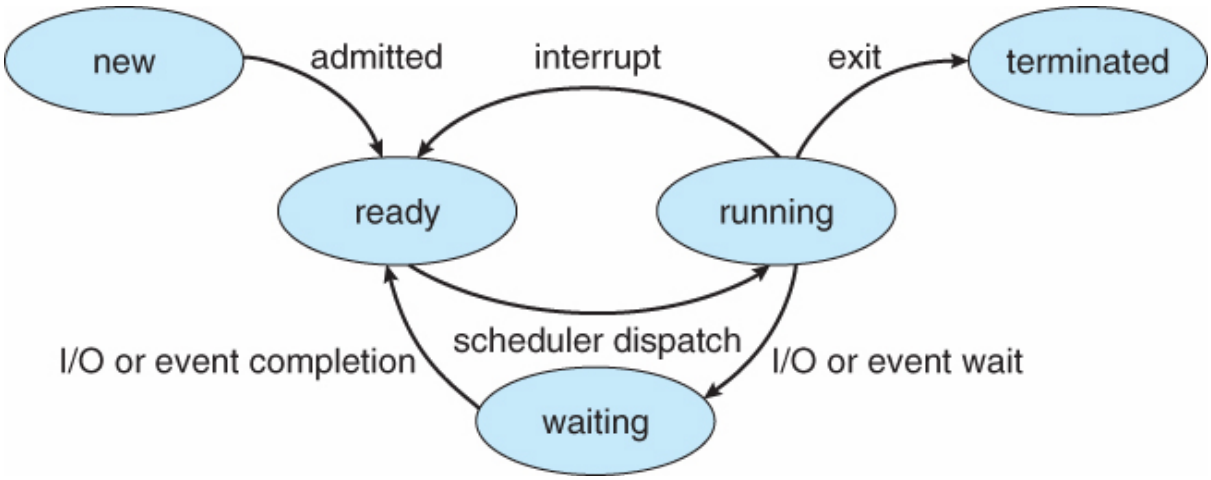


Figure 3.2 - Diagram of process state

3.1.3 Process Control Block

For each process there is a Process Control Block, PCB, which stores the following (types of) process-specific information, as illustrated in Figure 3.1. (Specific details may vary from system to system.)

- **Process State** - Running, waiting, etc., as discussed above.
- **Process ID**, and parent process ID.
- **CPU registers and Program Counter** - These need to be saved and restored when swapping processes in and out of the CPU.
- **CPU-Scheduling information** - Such as priority information and pointers to scheduling queues.
- **Memory-Management information** - E.g. page tables or segment tables.
- **Accounting information** - user and kernel CPU time consumed, account numbers, limits, etc.

- **I/O Status information** - Devices allocated, open file tables, etc.

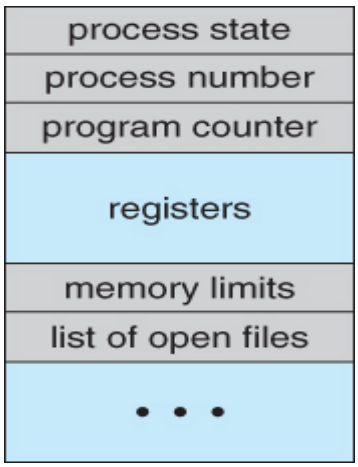
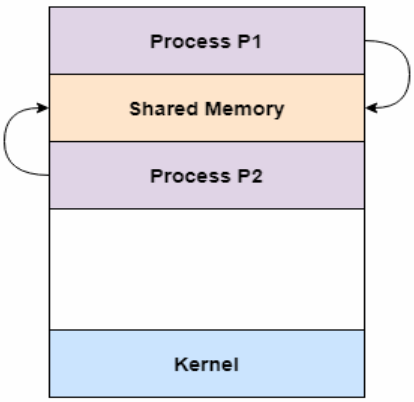


Figure 3.3 - Process control block (PCB)

Inter Process Communication (IPC)

- Processes need to communicate with each other in many situations. **Inter-Process Communication or IPC** is a mechanism that allows processes to communicate.
  - It helps processes synchronize their activities, share information and avoid conflicts while accessing shared resources.
  - There are two method of IPC, shared memory and message passing. An operating system can implement both methods of communication.

Shared Memory:
Communication between processes using shared memory requires processes to share some variable and it completely depends on how the programmer will implement it. Processes can use shared memory for extracting information as a record from another process as well as for delivering any specific information to other processes.



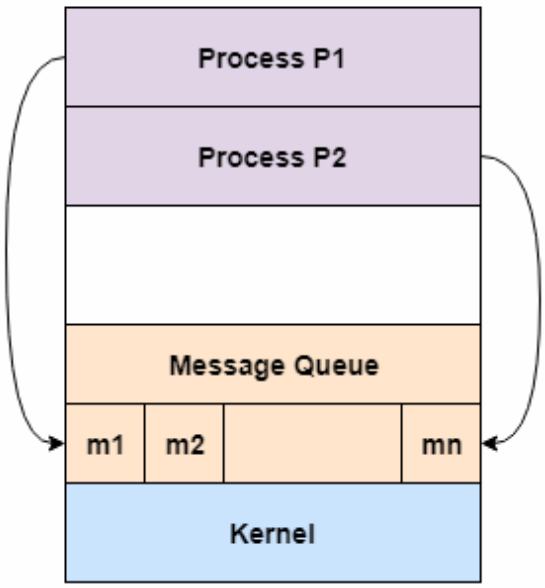
Shared Memory Model

- In the above shared memory model, a common memory space is allocated by the kernel.

- Process A writes data into the shared memory region (Step 1).
- Process B can then directly read this data from the same shared memory region (Step 2).
- Since both processes access the same memory segment, this method is fast but requires synchronization mechanisms (like semaphores) to avoid conflicts when multiple processes read/write simultaneously.

Message Passing:
Message Passing is a method where processes communicate by sending and receiving messages to exchange data.

- In this method, one process sends a message and the other process receives it, allowing them to share information.
- Message Passing can be achieved through different methods like Sockets, Message Queues or Pipes.



Message Passing Model

- In the above message passing model, processes exchange information by sending and receiving messages through the kernel.
- Process A sends a message to the kernel (Step 1).
- The kernel then delivers the message to Process B (Step 2).
- Here, processes do not share memory directly. Instead, communication happens via system calls (send(), recv(), or similar).
- This method is simpler and safer than shared memory because there's no risk of overwriting shared data, but it incurs more overhead due to kernel involvement.

Operating Systems Structures:

- An operating system can be built using different structures depending on how its components are organized and how they interact inside the kernel. Its system structure acts as a blueprint that shows how the OS is arranged so it can be understood and adapted to different needs. Since OS design is complex, breaking it into smaller, manageable parts makes development easier. The structure defines the strategy used to integrate these components within the kernel.

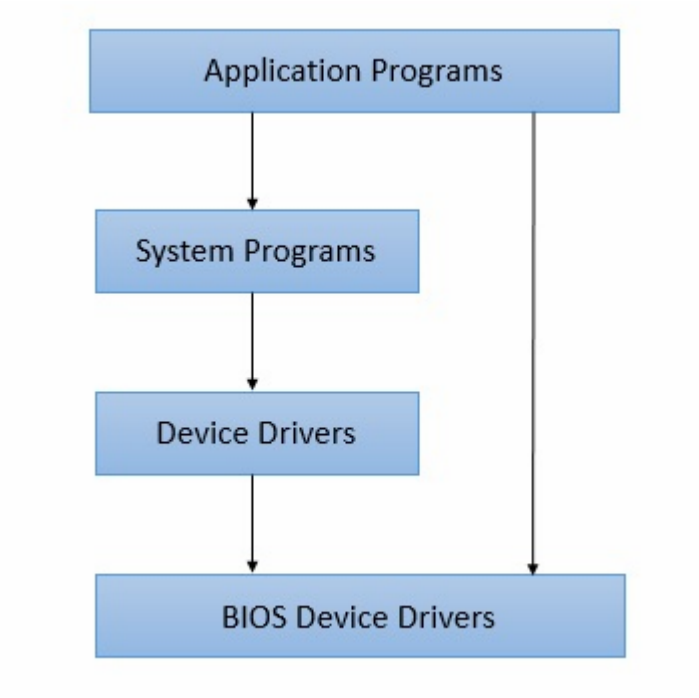
- OS structure depends on component organization and interaction.
- Acts as a blueprint that guides OS design.
- Breaking the OS into smaller parts simplifies development.
- Structure defines how components are integrated into the kernel.

Following are various popular implementations of Operating System structures.

- Simple Structure
- Monolith Structure
- Micro-Kernel Structure
- Exo-Kernel Structure
- Layered Structure
- Modular Structure
- Virtual Machines

Simple Structure

There are many operating systems that have a rather simple structure. These started as small systems and rapidly expanded much further than their scope. A common example of this is MS-DOS. It was designed simply for a niche amount for people. There was no indication that it would become so popular.



It is better that operating systems have a modular structure, unlike MS-DOS. That would lead to greater control over the computer system and its various applications. The modular structure would also allow the programmers to hide information as required and implement internal routines as they see fit without changing the outer specifications.

Advantages

Following are advantages of a simple operating system structure.

- **Easy Development** - In simple operation system, being very few interfaces, development is easy especially when only limited functionalities are to be delivered.
- **Better Performance** - Such a sytem, as have few layers and directly interects with hardware, can provide a better performance as compared to other types of operating systems.

Disadvantages

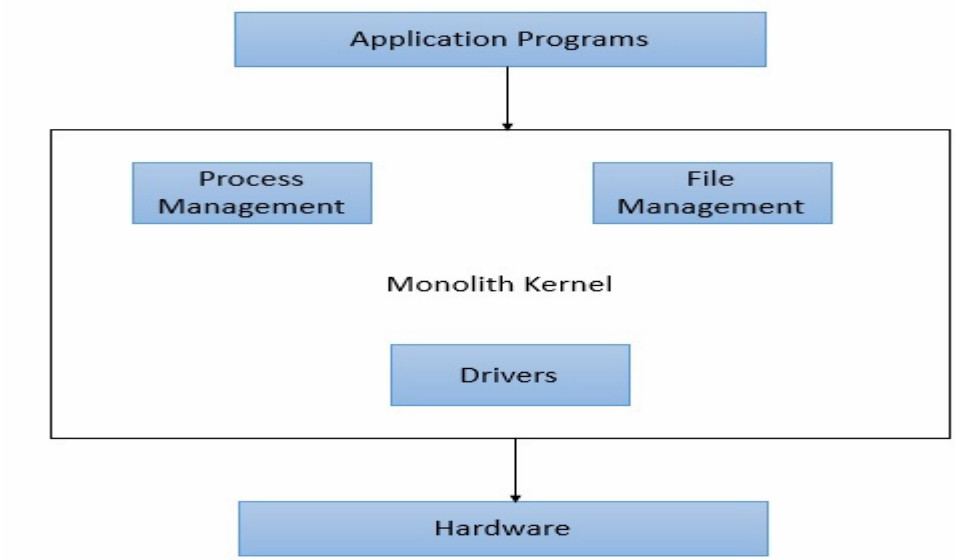
Following are disadvantages of a simple operating system structure.

- **Frequent System Failures** - Being poorly designed, such a system is not robust. If one program fails, entires operating system crashses. Thus system failures are quiet frequent in simple operating systems.
- **Poor Maintainability** - As all layers of operating systems are tightly coupled, change in one layer can impact other layers heavily and making code unmanageable over a period of time.

Monolith Structure:

In monolith structured operating system, a central piece of code called kernel is responsible for all major operations of an operating system. Such operations includes file management, memory management, device management and so on. The kernal is the main component of an operating system and it provides all the services of an operating system to the application programs and system programs.

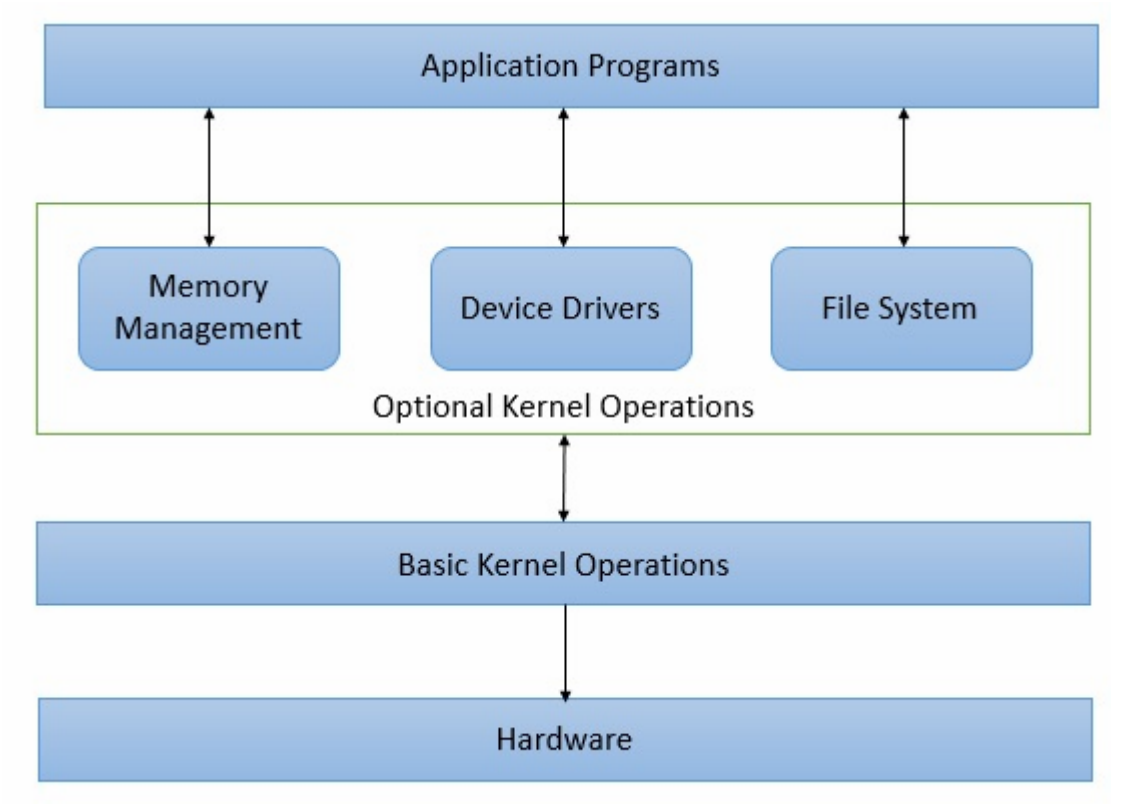
The kernel has access to the all the resources and it acts as an interface with application programs and the underlying hardware. A monolithic kernel structure promotes timesharing, multiprogramming model and was used in old banking systems.



Micro-Kernel Structure

As in case monolith structure, there was single kernel, in micro-kernel, we have multiple kernels each one specilized in particular service. Each microkernel is developed independent

to the other one and makes system more stable. If one kernel fails the operating sytem will keep working with other kernel's functionalities.



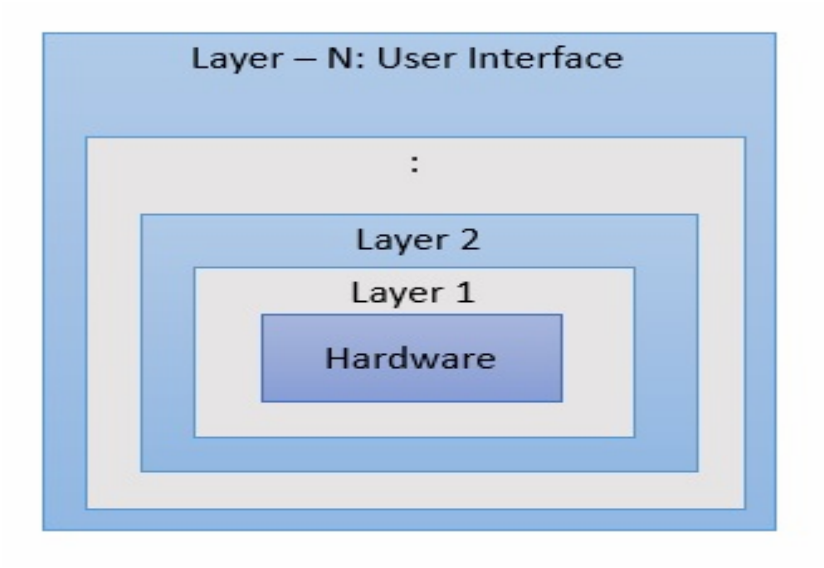
Exo-Kernel Structure

Exo-Kernal Structured operating system was designed and developed at MIT. The aim of this design was to keep Kernel size minimal while allowing the application programs to manage hardware resources directly. The purpose of removing abstraction of operating system for hardware resources was to enable application programmer to write high performance code while exo-kernel handles other operations.

Layered Structure

One way to achieve modularity in the operating system is the layered approach. In this, the bottom layer is the hardware and the topmost layer is the user interface.

An image demonstrating the layered approach is as follows –



As seen from the image, each upper layer is built on the bottom layer. All the layers hide some structures, operations etc from their upper layers.

One problem with the layered structure is that each layer needs to be carefully defined. This is necessary because the upper layers can only use the functionalities of the layers below them.

Modular Structure

Modular structure operating system works on the similar principle as a monolith but with better design. A central kernel is responsible for all major operations of operating system. This kernel has set of core functionality and other services are loaded as modules dynamically to the kernel at boot time or at runtime. Sun Solaris OS is one of the example of Modular structured operating system.

Virtual Machine Structure

In this kind of structure, hardware like CPU, memory, hard disks are abstracted into virtual machines. User can use them with actually configure them using execution contexts. Virtual machine takes a good amount of disk space and is to be provisioned. Muliple virtual machines can be created on a single physical machine.